

Principal Component Analysis (PCA) Explained in Matrix Notation

Introduction

Principal Component Analysis (PCA) is a dimensionality reduction technique that finds a low-dimensional representation of data that captures the most variance.

In this notebook, we explain PCA using **matrix notation**, step by step.

Step 1: Center the Data

Given a dataset with n samples and d features, represented by a matrix $X \in \mathbb{R}^{n \times d}$, we first subtract the mean of each column to center the data:

$$\bar{x} = \frac{1}{n} \sum_{i=1}^n x_i, \quad X_c = X - \mathbf{1}\bar{x}^\top$$

```
In [1]: import numpy as np

# Create a toy dataset
X = np.array([[2.5, 2.4],
              [0.5, 0.7],
              [2.2, 2.9],
              [1.9, 2.2],
              [3.1, 3.0],
              [2.3, 2.7],
              [2.0, 1.6],
              [1.0, 1.1],
              [1.5, 1.6],
              [1.1, 0.9]])

# Center the data
X_mean = np.mean(X, axis=0)
X_c = X - X_mean

X_c
```

```
Out[1]: array([[ 0.69,  0.49],
               [-1.31, -1.21],
               [ 0.39,  0.99],
               [ 0.09,  0.29],
               [ 1.29,  1.09],
               [ 0.49,  0.79],
               [ 0.19, -0.31],
               [-0.81, -0.81],
               [-0.31, -0.31],
               [-0.71, -1.01]])
```

Step 2: Compute the Covariance Matrix

Definition of scalar covariance

Suppose $y_1, y_2, \dots, y_n; z_1, z_2, \dots, z_n$ are scalar measurements

Let $\mu_y = \frac{1}{n} \sum_{i=1}^n y_i$; $\mu_z = \frac{1}{n} \sum_{i=1}^n z_i$ be the averages of y and z .

Then the covariance of y and z is

$$\text{Cov}(y, z) = \frac{1}{n} \sum_{i=1}^n (y_i - \mu_y)(z_i - \mu_z) = E(xy) - E(x)E(y)$$

The covariance matrix

The centered data can be seen as a column of n row vectors:

$$X_c = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix}$$

The empirical covariance matrix of data $x_1, x_2, \dots$ is

$$C = \frac{1}{n} \sum_{i=1}^n x_i^\top x_i = \frac{1}{n} X_c^\top X_c$$

```
In [2]: # Compute the covariance matrix
n_samples = X_c.shape[0]
C = (X_c.T @ X_c) / n_samples
C
```

```
Out[2]: array([[0.5549, 0.5539],
               [0.5539, 0.6449]])
```

Step 3: Eigen Decomposition

We compute the eigenvalues and eigenvectors of the covariance matrix:

$$Cv = \lambda v$$

The eigenvectors are the principal components. Sorting by decreasing eigenvalue gives the directions of maximum variance.

```
In [3]: # Eigen decomposition
eigvals, eigvecs = np.linalg.eigh(C)

# Sort by eigenvalue in descending order
idx = np.argsort(eigvals)[::-1]
eigvals = eigvals[idx]
eigvecs = eigvecs[:, idx]

eigvals, eigvecs
```

```
Out[3]: (array([1.15562494, 0.04417506]),
        array([[ 0.6778734 , -0.73517866],
               [ 0.73517866,  0.6778734 ]]))
```

Step 4: Project the Data

To reduce dimensionality to k components, project the centered data onto the top k eigenvectors:

$$Z = X_c V_k$$

```
In [4]: # Project onto the first principal component
k = 1
V_k = eigvecs[:, :k]
Z = X_c @ V_k

Z
```

```
Out[4]: array([[ 0.82797019],
               [-1.77758033],
               [ 0.99219749],
               [ 0.27421042],
               [ 1.67580142],
               [ 0.9129491 ],
               [-0.09910944],
               [-1.14457216],
               [-0.43804614],
               [-1.22382056]])
```

Step 5: Reconstruction from Principal Components

We can approximately reconstruct the original data from the lower-dimensional representation:

$$\hat{X} = ZV_k^\top + \bar{x}$$

```
In [5]: # Reconstruct from the projection
X_reconstructed = Z @ V_k.T + X_mean
X_reconstructed
```

```
Out[5]: array([[2.37125896, 2.51870601],
               [0.60502558, 0.60316089],
               [2.48258429, 2.63944242],
               [1.99587995, 2.11159364],
               [2.9459812 , 3.14201343],
               [2.42886391, 2.58118069],
               [1.74281635, 1.83713686],
               [1.03412498, 1.06853498],
               [1.51306018, 1.58795783],
               [0.9804046 , 1.01027325]])
```

2D Visualization of PCA

We now visualize the original data and its projection onto the first principal component. This helps us understand how PCA reduces dimensionality.

```
In [6]: import matplotlib.pyplot as plt

# Plot original data
plt.figure(figsize=(8, 6))
plt.scatter(X_c[:, 0], X_c[:, 1], label='Original Data', alpha=0.6)

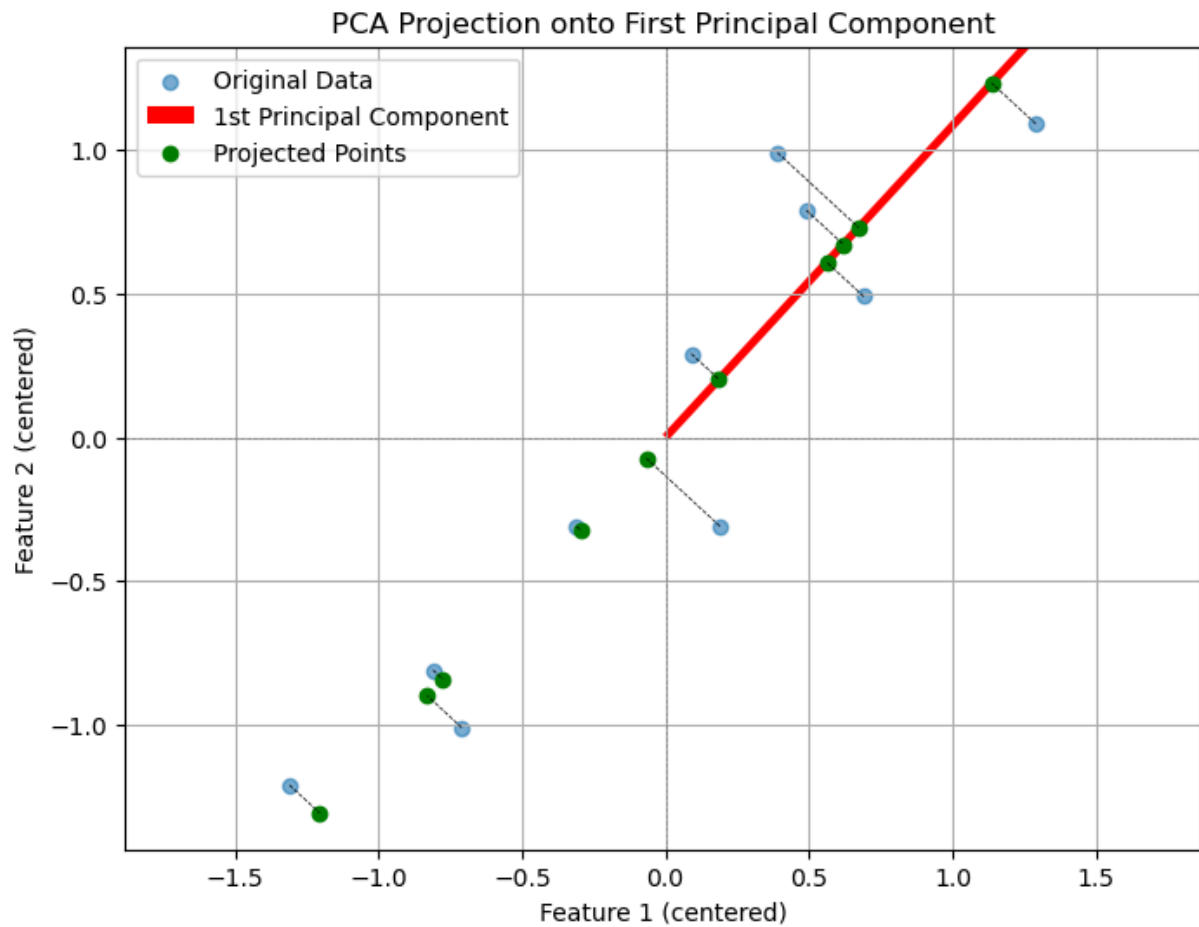
# Plot principal component direction
origin = np.mean(X_c, axis=0)
scale = 3 # for visualization
plt.quiver(*origin, *eigvecs[:, 0], scale=1/scale, color='red', label='1st F

# Projected points (back into original space)
X_proj = Z @ V_k.T
plt.scatter(X_proj[:, 0], X_proj[:, 1], color='green', label='Projected Poir

# Draw projection lines
for i in range(len(X_c)):
    plt.plot([X_c[i, 0], X_proj[i, 0]], [X_c[i, 1], X_proj[i, 1]], 'k--', li

plt.axhline(0, color='gray', linestyle='--', linewidth=0.5)
plt.axvline(0, color='gray', linestyle='--', linewidth=0.5)
plt.legend()
plt.title("PCA Projection onto First Principal Component")
plt.xlabel("Feature 1 (centered)")
plt.ylabel("Feature 2 (centered)")
plt.grid(True)
```

```
plt.axis('equal')  
plt.show()
```



In []: